ccdisk: A High-Performance, Deterministic Python 2.7 Disk Analyzer and Storage Management Utility

Dr. Travis Lee Thompson*
Independent Systems Research Laboratory

November 2026

Abstract

Disk space management in constrained or legacy computing environments requires high-performance, deterministic execution without risking stack overflow or unintended filesystem modifications. We present `ccdisk`, a modular, non-recursive disk analysis and storage utility implemented in Python 2.7 [2]. `ccdisk` incorporates iterative queue-based directory traversal, inode-level hard link de-duplication, sparse file block allocation metrics, squarified treemap layout algorithms [1, 3], automated build artifact and cache detection, and staged cleanup sets protected by rigorous path safety barriers and operating system Trash migration.

1 Introduction

Disk visualization tools frequently depend on recursive directory walkers that risk stack exhaustion on deep hierarchies or fail to de-duplicate hard-linked inodes, leading to inflated storage reporting. In specialized computing environments—such as offline legacy systems, embedded telemetry diagnostic stations, and deterministic runtimes—administrators require transparent tools that adhere to strict safety ground rules: zero recursion, static memory allocation patterns, explicit runtime assertions, and non-destructive cleanup workflows.

2 Core Architecture

`ccdisk` is structured into twelve modular subsystems mirroring systems engineering best practices:

1. **Iterative Scanner:** Non-recursive queue walker handling permission exceptions gracefully without halting traversal.
2. **Storage Metrics Engine:** Distinguishes apparent file size from actual allocated disk blocks ($st_blocks \times 512$), correctly handling sparse files.
3. **Inode De-duplication & Symlink Safety:** Tracks device and inode tuples ($\text{st_dev}, \text{st_ino}$) to ensure hard-linked files share storage costs exactly once, while avoiding symlink target traversal loops.
4. **Squarified Treemap Layouts:** Organizes hierarchical items into aspect-ratio optimized 2D rectangles [3].

*Independent Systems Researcher & Software Author. ORCID: 0009-0001-7917-0306

5. **Interactive Navigation & Inspection:** Manages breadcrumb stacks, scope zooming, metadata inspection, and largest-children rankings.

6. **Reclaimable Detection & Staging:** Automatically flags build directories (`node_modules`, `target`, `__pycache__`) and user caches, supporting staged cleanup sets with duplicate protection and volume free space predictions.

7. **Safety Barriers & Trash Migration:** Enforces strict path protection guards (blocking root directories and paths outside scan scopes) and executes recoverable migrations to the operating system Trash.

3 Squarified Treemap Mathematics

Following the squarified layout methodology [1], given rectangular areas proportional to file disk usage s_i, the aspect ratio r for a row of width w and total area s is minimized by evaluating:

$$r = \max\left(\frac{w^2 \cdot \max(s_i)}{s^2}, \frac{s^2}{w^2 \cdot \min(s_i)}\right) \tag{1}$$

This mathematical optimization prevents unreadable, elongated slivers when rendering directories containing thousands of small files.

4 Conclusion

`ccdisk` provides a robust, fully deterministic storage management solution for Python 2.7 environments [2]. By combining rigorous safety assertions, hard link de-duplication, and recoverable trash migration, it bridges the gap between high-performance system telemetry and safe disk administration.

References

[1] Mark Bruls, Kees Huizing, and Jarke J. van Wijk. Squarified treemaps. In *Joint Eurographics and IEEE TCVG Symposium on Visualization*, pages 33–42. IEEE Computer Society, 2000.

[2] Wesley J. Chun. *Core Python Programming.* Prentice Hall Professional, 2nd edition, 2006.

[3] Brian Johnson and Ben Shneiderman. Tree-maps: A space-filling approach to the visualization of hierarchical information structures. *IEEE Visualization*, 2:284–291, 1991.